How I Added a Custom System Call to the Linux Kernel — A Complete Step-by-Step Guide

Ever wondered how your program actually talks to the OS kernel? I built a custom Linux system call from scratch — and it worked. Here's the full guide for Kernel 6.11 so you can do it too.

System calls are the bridge between user-space applications and the kernel. When your code opens a file, allocates memory, or sends a network packet — it goes through a system call. In this guide, I'll walk you through creating your own custom syscall from zero to a working test program.

What is a System Call?

A system call (syscall) is the mechanism by which a user-space program requests a service from the operating system's kernel. When a program needs privileged access — like reading a file, allocating memory, or sending data over a network — it can't do that directly. Instead, it makes a system call, which transfers control to the kernel. The kernel performs the operation on the program's behalf, then returns the result.

Modern operating systems use two privilege levels:

- **User space** — where regular applications run, with restricted access to hardware and memory.
- **Kernel space** — where the OS core runs, with full hardware access. This separation protects the system from buggy or malicious programs.

How a System Call Works (Flow)

Here's the execution flow when your program triggers a custom syscall:

1. Program calls syscall(463)
2. CPU switches to kernel mode
3. Kernel looks up sys_hello in the syscall table at index 463
4. Your kernel function runs → prints to kernel log
5. Returns 0 back to your program
6. You see: "System call returned: 0" ✓

© 2026 Linux Kernel Foundation (LKF)

Complete Guide: Adding a Custom Syscall (Kernel 6.11)

Step 1: Navigate to Kernel Source

Go to your kernel source directory:

```
cd /usr/src/linux-source-6.11.0
```

Step 2: Create Your Syscall Implementation

Create a new directory and implement your syscall:

```
mkdir hello
vi hello/hello.c

#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(hello)
{
   pr_info("My System call implementation\n");
   return 0;
}
```

Step 3: Create the Makefile

Create hello/Makefile with the following content:

```
vi hello/Makefile
obj-y := hello.o
```

Step 4: Modify the Top-Level Makefile

Find core-y (around line 730) and add hello/ to it:

```
# Before:
core-y +=

# After:
core-y += hello/
```

Step 5: Add the System Call Number

Open the syscall table and add your entry at the end (check the highest existing number, then use the next one — e.g., 463):

```
vi arch/x86/entry/syscalls/syscall_64.tbl


463    common    hello    sys_hello
```

© 2026 Linux Kernel Foundation (LKF)

Step 6: Declare the Syscall Prototype

Add the prototype to syscalls.h, before the #endif line:

```
vi include/linux/syscalls.h
asmlinkage long sys_hello(void);
```

Step 7: Configure and Build the Kernel

```
# Copy current kernel config
cp /boot/config-$(uname -r) .config
make olddefconfig

# In .config, set these to empty strings:
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_REVOCATION_KEYS=""
CONFIG_MODULE_SIG_KEY=""

# Build and install
make -j$(nproc)
sudo make modules_install
sudo make install
sudo update-grub
sudo reboot
```

At boot: select your new kernel from GRUB (under "Advanced options" if needed).

Step 8: Test Your System Call

After rebooting, verify your kernel version, then create and compile a test program:

```
uname -r   # Should show 6.11.0

// test_syscall.c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define __NR_hello 462

int main() {
    long result = syscall(__NR_hello);
    if (result == 0)
        printf("System call succeeded!\n");
    return 0;
}
```

© 2026 Linux Kernel Foundation (LKF)

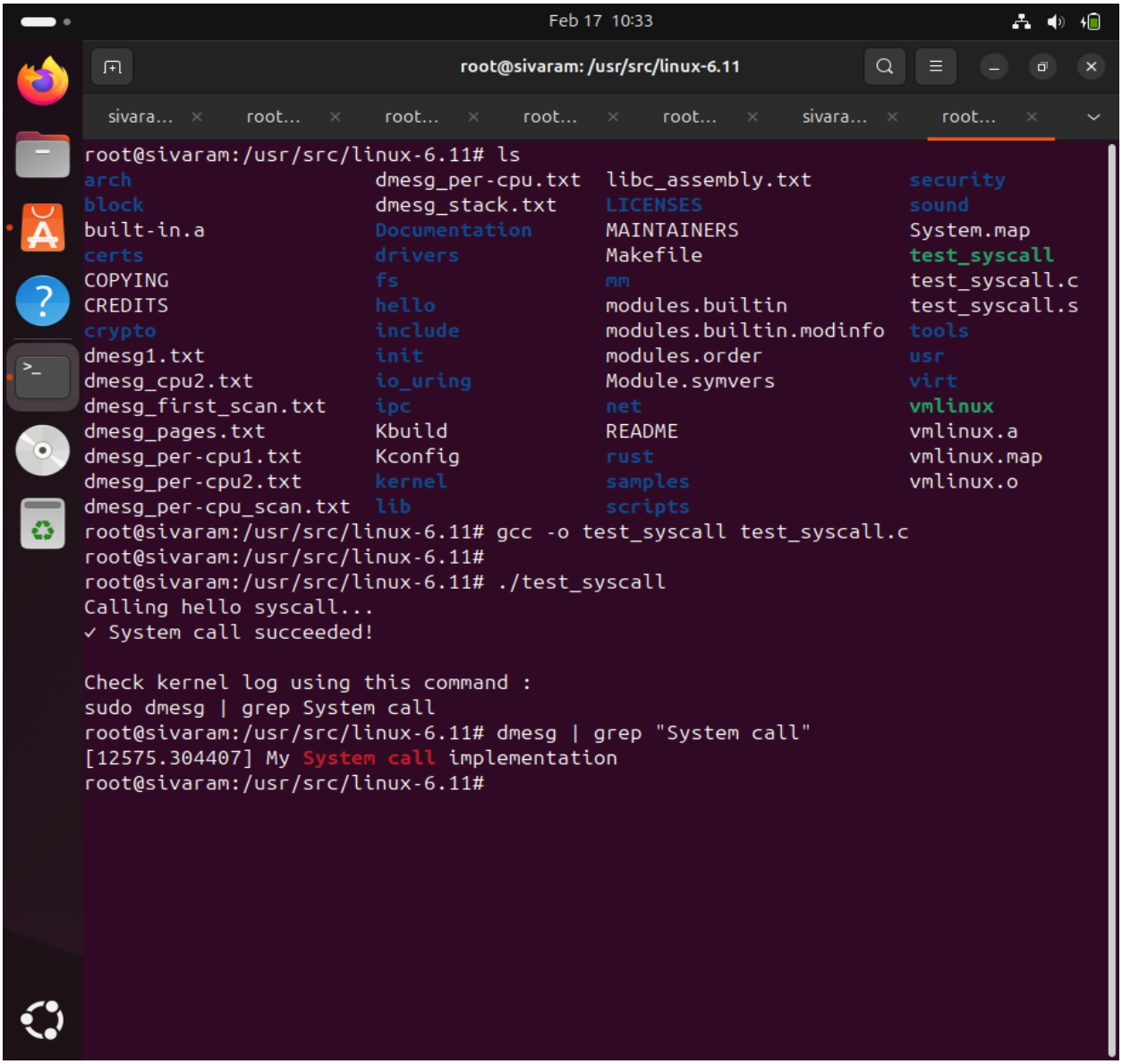
```
#Compile the test_syscall.c file.
gcc -o test_syscall test_syscall.c
./test_syscall

# Check kernel log
sudo dmesg | grep "System call"
```

Expected output:

```
System call succeeded!
[12345.678901] My System call implementation
```

Here is the actual proof — the syscall working live on my machine:



System call succeeded! Kernel log confirms: My System call implementation

Key Takeaways

- System calls are the secure gateway between user programs and the kernel.
- Every syscall has a unique integer ID registered in a syscall table.
- Building and booting a custom kernel is fully achievable — even as a learning exercise.
- Always disable signature key checks in .config when building a test kernel.

Have you ever worked with kernel development or system programming? I'd love to hear your experience in the comments below!

#Linux #Kernel #SystemProgramming #C #OpenSource #LowLevelProgramming #OperatingSystems #SoftwareEngineering #DevCommunity

© 2026 Linux Kernel Foundation (LKF)